# BHASKARACHARYA NATIONAL INSTITUTE FOR SPACE APPLICATIONS AND GEO-INFORMATICS

**WEEKLY PROGRESS REPORT (26/10/2026 – 01/11/2026)**

**WEEK 14**

| PROJECT NAME | CHATBOT FOR STRUCTURED, UNSTRUCTURED AND SEMANTIC KNOWLEDGE QUERY |
|---|---|

| | |
|---|---|
| **PROJECT DESCRIPTION :** | DESIGN AND IMPLEMENT A CHATBOT THAT CAN ANSWER TO DIFFERENT TYPE OF QUERY LIKE STRUCTURED, UNSTRUCTURED AND SEMANTIC KNOWLEDGE BY DECIDING OWN WHAT TYPE OF QUERY BY |
| **GROUP MEMBER :** | PRATYUSH ANAND, ADITYA SINGH |
| **GROUP ID :** | 2026G306 |
| **GROUP GUIDE :** | VEDANT JOSHI |
| **GROUP COORDINATOR :** | SIDHDHARTH PATEL |
| **COLLEGE GUIDE NAME :** | EHT SHAM |
| **COLLEGE GUIDE No. :** | +91 9818938592 |
| **COLLEGE GUIDE EMAIL. :** | Ehte.sham@pilani.bits-pilani.ac.in |
| **COLLEGE NAME :** | BITS PILANI |

| | |
|---|---|
| **26/10/2026** | • **Fixed and froze the prior structured-request interpretation work; the final version passed 589 Python tests, 8 frontend checks and 16 real-model development checks.**<br>• **Confirmed the preserved original evaluation remained unchanged: N03/N06 correct, N01/N04/N05 failed, N02 required clarification.** |
| **27/10/2026** | • Repaired shared SQL interpretation so multiple aggregations retain separate field bindings (e.g. minimum price, maximum price and average carat).<br>• Made conditional counts and percentages preserve numerator, denominator and shared-filter scopes. |
| **28/10/2026** | • Represented Boolean OR conditions correctly so overlapping rows are counted once; requests to retain zero-match groups now use conditional aggregation instead of dropping groups through WHERE.<br>• Reduced contextual words being mistaken for category values while explicit values such as clarity = IF still work; preserved organization isolation, selected-file restrictions, source/version evidence, read-only SQL, NULL behavior, citations and trace diagnostics. |
| **29/10/2026** | • Started the first-release readiness pass and added a capability review document (first_release_review.md).<br>• Identified that request-contract construction was still distributed across several parsers and stages. |
| **30/10/2026** | • Added an authoritative contract builder that reconciles proposals, retains provenance, preserves distinct scoped measures and seals the contract before SQL generation.<br>• Added focused regression coverage for the contract builder; 68 targeted tests passed.<br>• Began setting up a bounded cross-format development verification (CSV, Excel, PDF, DOCX, text, images, semantic retrieval, selected-file scope, follow-ups, multi-source questions, organization isolation); not yet completed or frozen.<br>• Attempted real-browser verification, but no browser surface was available, so browser checks remain a release blocker; found a manifest for a larger local model, but its availability and practical execution are not yet confirmed. |

| | |
|---|---|
| **31/10/2026** | **Holiday** |
| **01/11/2026** | **Holiday (Sunday)** |

| | |
|---|---|
| **Next Week Plan** | Finish the isolated cross-format verification, collect the model-comparison result or blocker, document browser unavailability, produce the release report and freeze that exact version (without creating or running a new unseen evaluation batch). |

## REFERENCE:


https://www.sqlite.org/lang_aggfunc.html

https://www.sqlite.org/lang_expr.html#case

https://www.sqlite.org/lang_select.html

https://docs.pytest.org/en/stable/

https://langchain-ai.github.io/langgraph/

https://docs.ollama.com/

https://docs.pydantic.dev/latest/


## SCREEN SHOTS:

```python
def run_performance_eval(examples: list[GoldenExample]) -> dict[str, Any]:
    """Runs the full agent graph once per example and records latencies.

    Returns both an aggregate PerformanceMetrics (P50 latency as the
    headline number) and the raw per-question latencies so callers (e.g.
    the API route) can report P95 too.
    """
    latencies_ms: list[float] = []
    tool_latencies_ms: list[float] = []

    for example in examples:
        start = time.perf_counter()
        with tenant_scope(example.tenant_id):
            graph = get_agent_graph()
            graph.invoke(new_state(example.tenant_id, example.question))
        latencies_ms.append((time.perf_counter() - start) * 1000)

    p50 = _percentile(latencies_ms, 50)
    p95 = _percentile(latencies_ms, 95)

    return {
        "metrics": PerformanceMetrics(end_to_end_latency_ms=p50),
        "p50_latency_ms": p50,
        "p95_latency_ms": p95,
        "mean_latency_ms": statistics.mean(latencies_ms) if latencies_ms else 0.0,
        "sample_count": len(latencies_ms),
        "raw_latencies_ms": latencies_ms,
    }
```